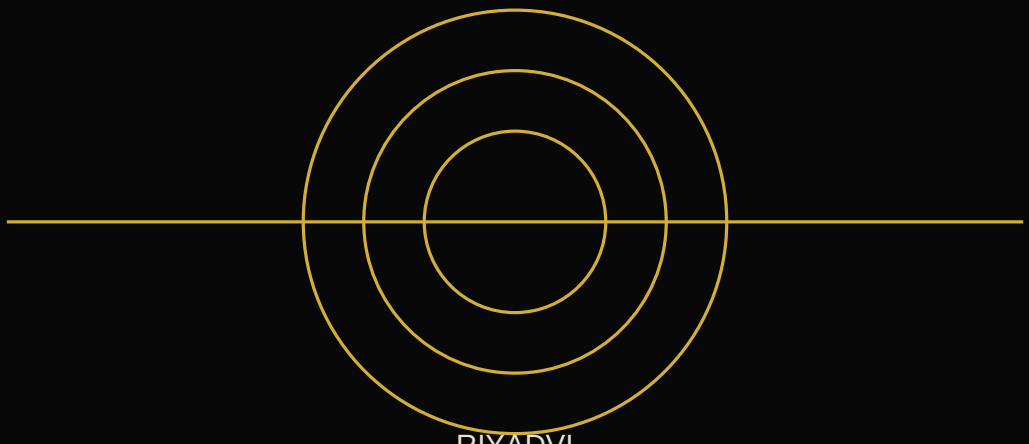

RIYADVI

SOFTWARE PROJECT PLANNING GUIDE

A practical framework to turn a business idea into a clear, buildable software plan.

DISCOVER • DEFINE • DESIGN • BUILD • LAUNCH

Start with clarity, not code

Successful software projects begin with a shared understanding of the problem, the people who experience it, and the outcome the business needs. Use this guide with business owners, product leads, designers, and developers. Record decisions, assumptions, open questions, and owners.

1. Define the business problem

Complete these prompts:
- **The primary users are…**
- **They currently struggle to…**
- **The current process/tool is…**
- **This causes… (time, cost, errors, lost sales, or poor experience)**
- **We will consider the problem improved when…**

2. Set measurable goals

Choose a small number of outcomes that can be measured. Avoid goals such as “make the app better” unless you define what better means.

Goal	Metric	Baseline	Target
Reduce manual work	Hours spent per week	Record current value	Set a realistic target
Improve conversion	Completed enquiries / visits	Measure current rate	Define target rate
Improve reliability	Errors or failed tasks	Review recent data	Agree acceptable level

3. Identify users and journeys

List the main user groups and what each needs to accomplish. Map the steps from entry to successful completion, including where users may get stuck.

User journey worksheet
User type: _____
Main task: _____
Entry point: _____
Steps they take: _____
Potential friction: _____
Successful outcome: _____

4. Define the minimum viable product (MVP)

An MVP is the smallest useful release that can test the core value proposition. Separate launch-critical features from improvements that can follow.

Priority	Meaning	Examples
Must have	Required for the core workflow or launch	Login, primary task, data saving, essential notifications
Should have	High value, but not essential for first release	Advanced filters, exports, extra integrations
Could have	Useful if time and budget allow	Personalization, additional dashboards
Not now	Explicitly deferred to protect focus	Nice-to-have features without a clear outcome

5. Sketch before building

Create low-fidelity wireframes for important screens and a simple flow diagram for the key journey. Validate navigation, terminology, empty states, error messages, and mobile layouts before detailed visual design.

6. Choose a sensible technical approach

Select technologies based on product needs, team capability, maintainability, security, and cost—not novelty alone. Decide what should be built, configured, or integrated.

Architecture questions

- **Which clients are required: web, mobile, or both?**
- **What data must be stored, and who can access it?**
- **Which external systems or APIs are required?**
- **What authentication and permission levels are needed?**
- **What are the backup and recovery expectations?**
- **Who will maintain the product after launch?**

7. Plan privacy, security, and quality

Collect only the data needed, protect secrets in environment variables, use appropriate access controls, validate inputs on the server, and avoid exposing sensitive details in logs. Plan tests for core journeys, invalid inputs, permissions, responsive layouts, accessibility, performance, and failure states.

8. Estimate scope, timeline, and budget

Break work into small deliverables and estimate with the people who will do the work. Include discovery, design, development, review, testing, deployment, and contingency.

Phase	Typical activities	Exit condition
Discovery	Problem, users, goals, scope	Problem and MVP agreed
Design	Flows, wireframes, visual design	Key journeys reviewed
Build	Implementation, integrations, reviews	MVP features complete
Validate	Testing, fixes, user feedback	Release criteria met
Launch & learn	Deployment, monitoring, improvements	Metrics and owners active

9. Prepare for launch

Launch checklist

- ■ **Core user journeys work end to end**
- ■ **Forms validate input and show helpful errors**
- ■ **Access permissions and secrets are reviewed**
- ■ **Backups and recovery expectations are defined**
- ■ **Mobile and desktop layouts are checked**
- ■ **Performance and accessibility are reviewed**
- ■ **Analytics or product metrics are configured**
- ■ **Support contact and issue-reporting process are ready**
- ■ **Deployment and rollback steps are documented**
- ■ **Stakeholders approve the release**

10. Measure, learn, and improve

After launch, compare real usage with the goals you set. Review support requests, user feedback, task completion, performance, and business outcomes. Prioritize improvements based on evidence.

Project one-page brief

Project name: _____

Business owner: _____

Problem to solve: _____

Primary users: _____

Top three outcomes: _____

MVP features: _____

Out of scope: _____

Target launch date: _____

Budget range: _____

Key risks / dependencies: _____

Success metrics and owners: _____

Common mistakes to avoid

- Starting before the problem and success measures are clear.
- Trying to deliver every feature in the first release.
- Leaving security, accessibility, or mobile support until the end.
- Treating estimates as guarantees instead of assumptions.
- Launching without ownership for support and maintenance.

A strong plan does not remove every uncertainty. It makes important uncertainties visible and helps everyone agree on what success looks like.

Prepared by Riyadvi Software Technologies | Custom software, digital experiences, and business-focused technology.